

Multi-Scale Protein Structure Modelling with Geometric Graph U-Nets

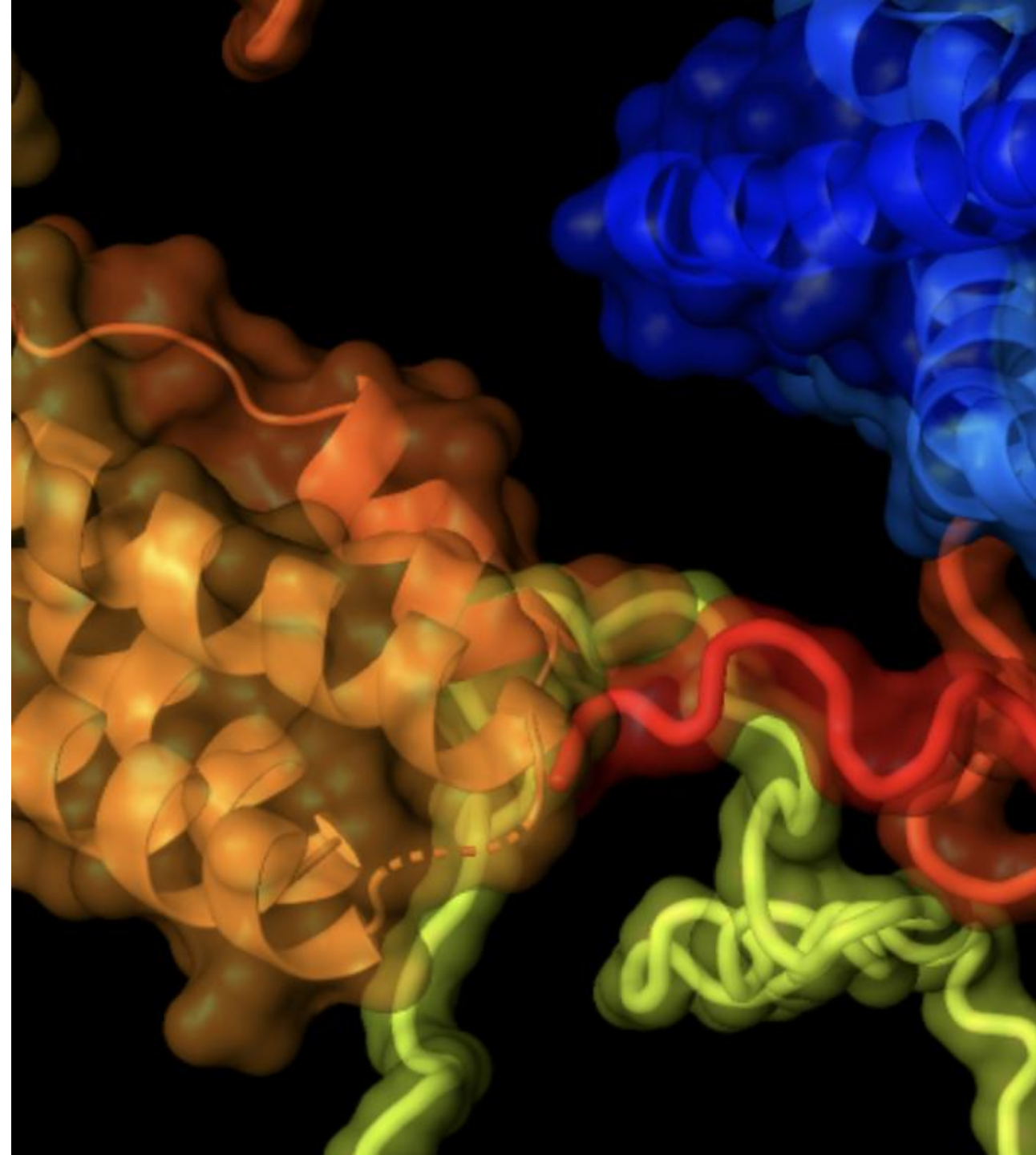
Authors: Chang Liu*, Vivian Li*, Linus Leong, Vladimir Radenkovic,
Pietro Liò, Chaitanya K. Joshi

Date: 01-03-2026

Speaker: Chang Liu

Agenda

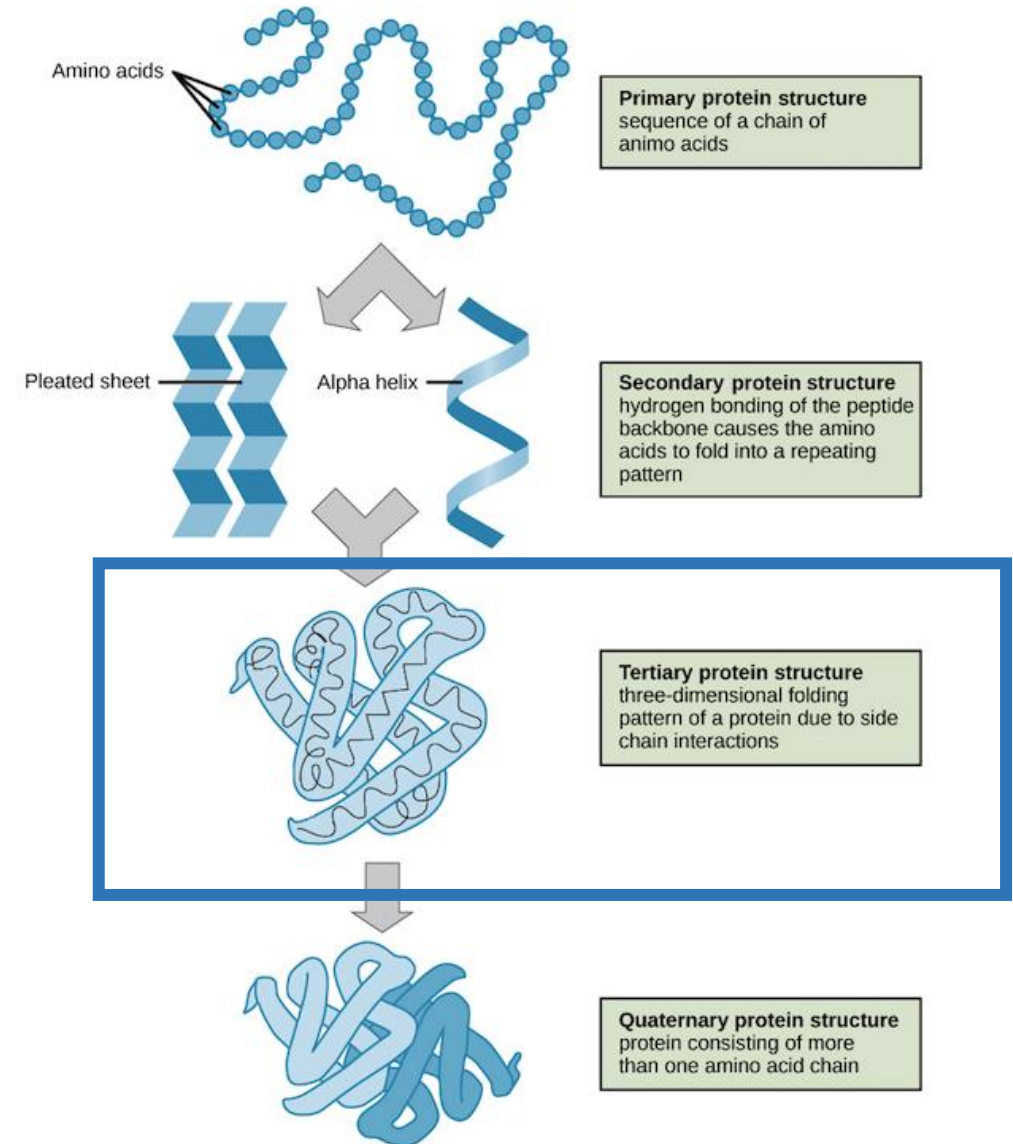
- **Background**
- Problem to Solve
- Method
- Experiments
- Theoretical Proof
- Conclusion & Discussion



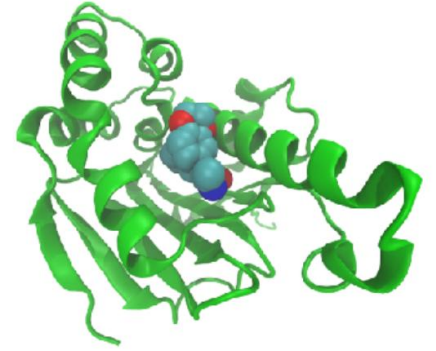
Background

Protein Fold Classification

- The task of grouping proteins into categories based on **the overall three-dimensional arrangement of their secondary-structure elements** (α -helices and β -sheets).



Why Protein Fold Classification?



Regarding the task itself

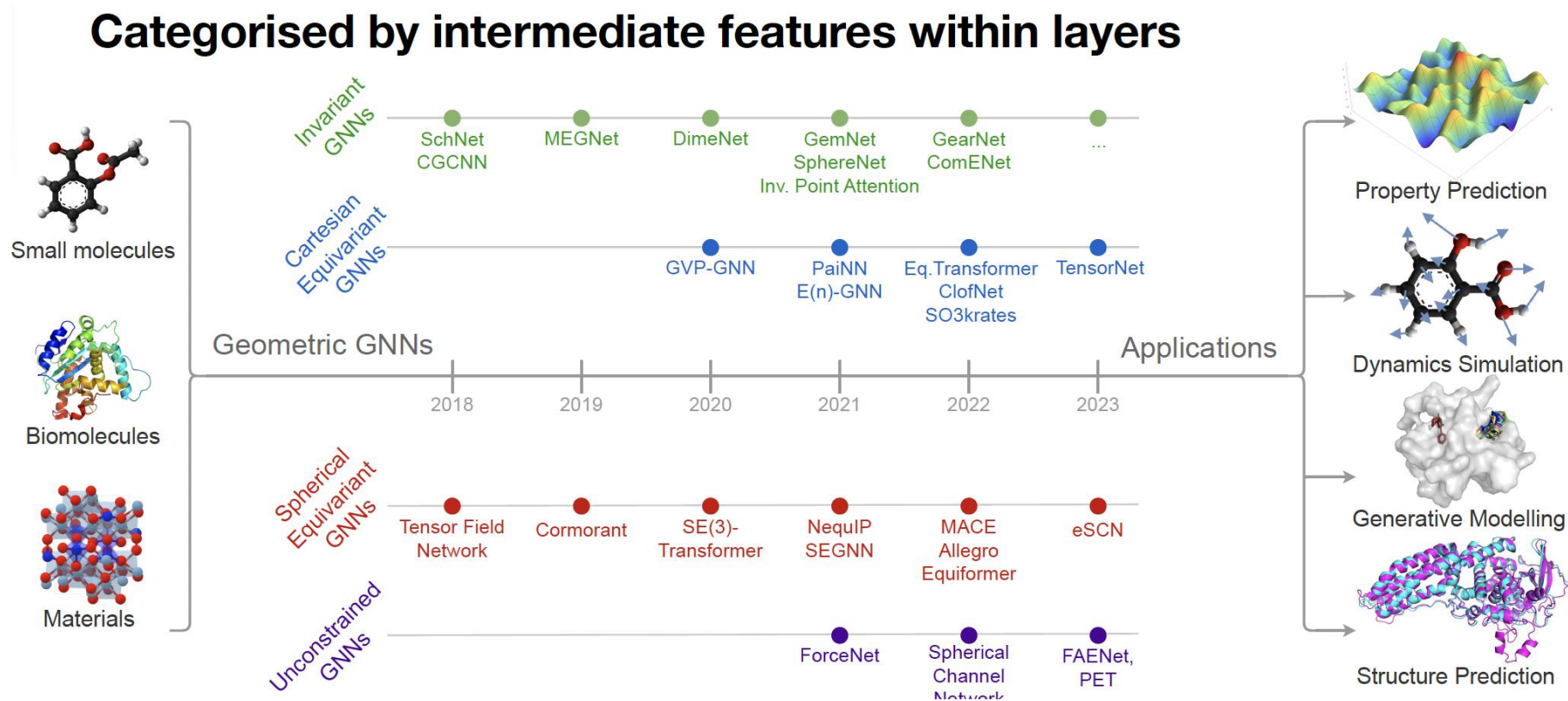
1. Function inference: similar folds often imply similar biochemical roles
2. Evolutionary insight: reveals distant homologs
3. Structure prediction & annotation: helps label new or predicted structures

Regarding our study

- Protein fold classification serves as a litmus test for the ability of a model to distinguish different structural folds.
- Models that perform poorly on distinguishing fold classes likely learn limited or low-quality structural representations. [38]
- The capability of learning the structural representations can be extended to other relevant tasks, such as inverse folding.

Background

There emerged a lot of approaches based on **geometric graph neural network** and **transformers** achieving good performance in different applications, such as protein fold classification and protein structure prediction.



AlphaFold

What does AlphaFold do?

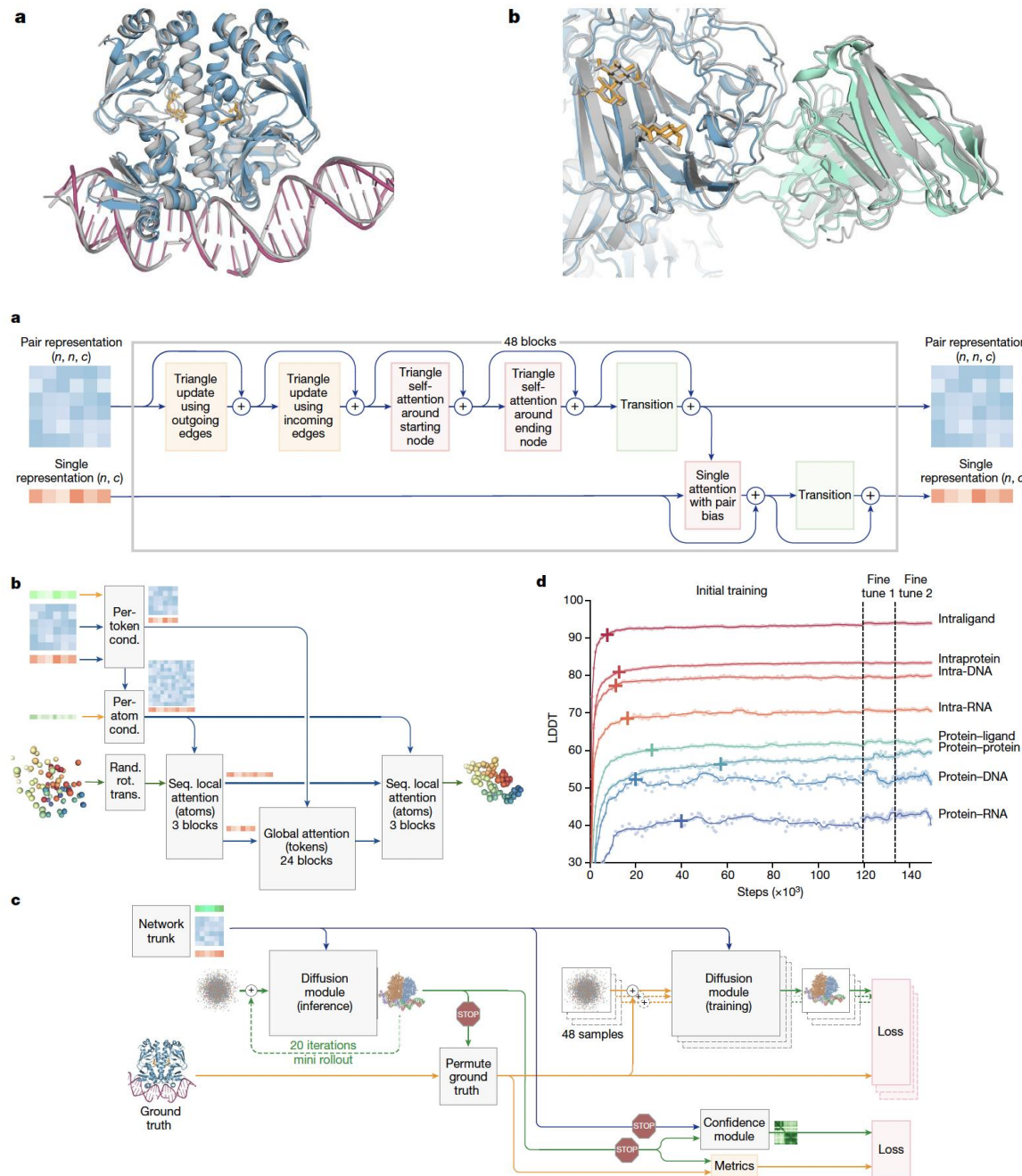
- AlphaFold solved the protein structure prediction **at scale**—predicting accurate 3D protein structures directly from amino-acid sequences.

Why is AlphaFold so impressive?

AlphaFold significantly proved the effectiveness of:

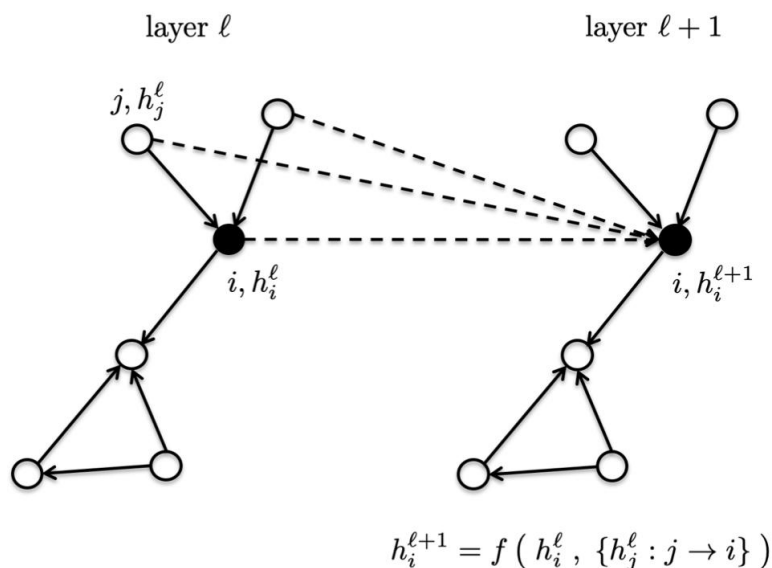
End-to-end Neural Nets -> Protein Structure Prediction

which then boosts protein fold classification, functional annotation, drug discovery.

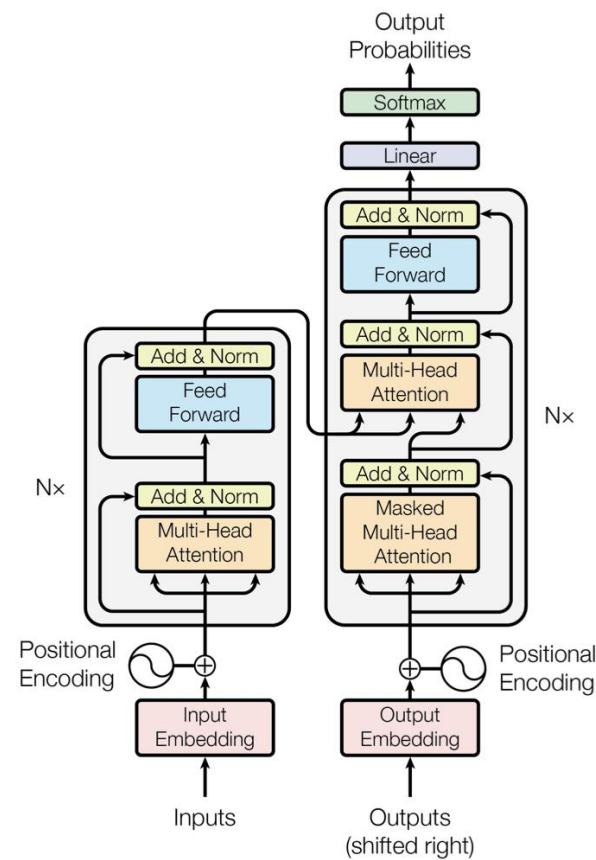


Problem to Solve

- Geometric Graph Neural Networks (GNNs) and Transformers have become state-of-the-art for learning from 3D protein structures.



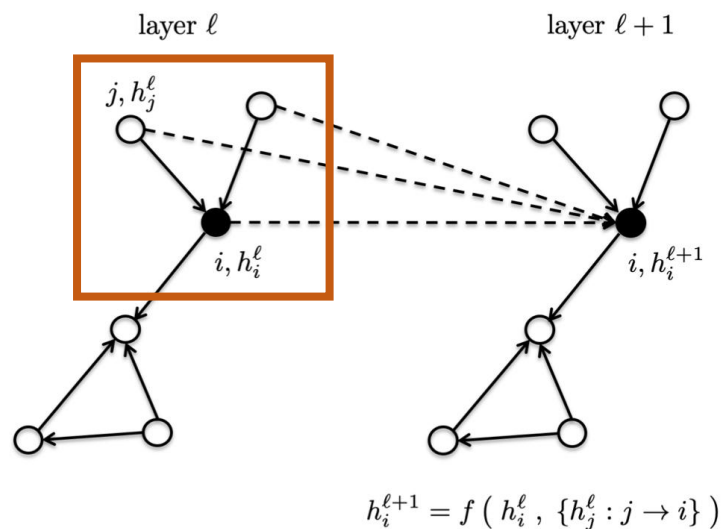
a Geometric GNNs



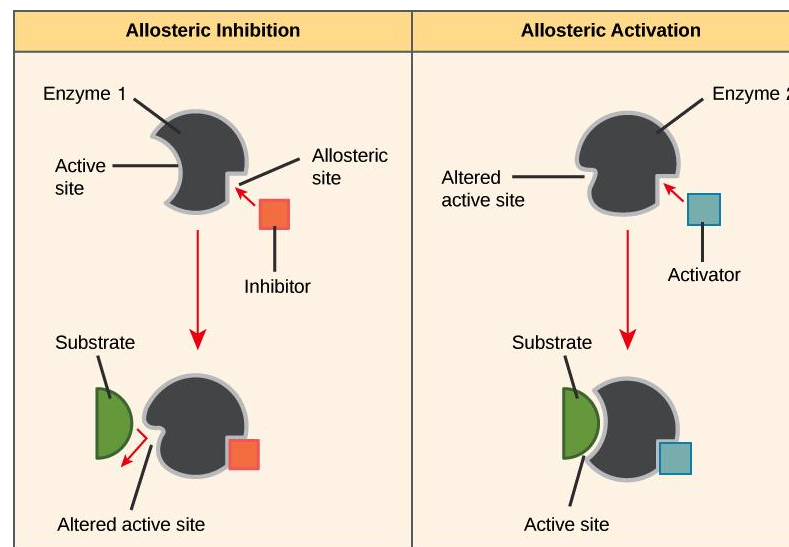
b Transformer

Problem to Solve

- The reliance of previous geometric GNNs on message passing prevents them from capturing the hierarchical interactions that govern protein function, such as global domains and long-range allosteric regulation.
- Transformer-based models consume significant computational resources.



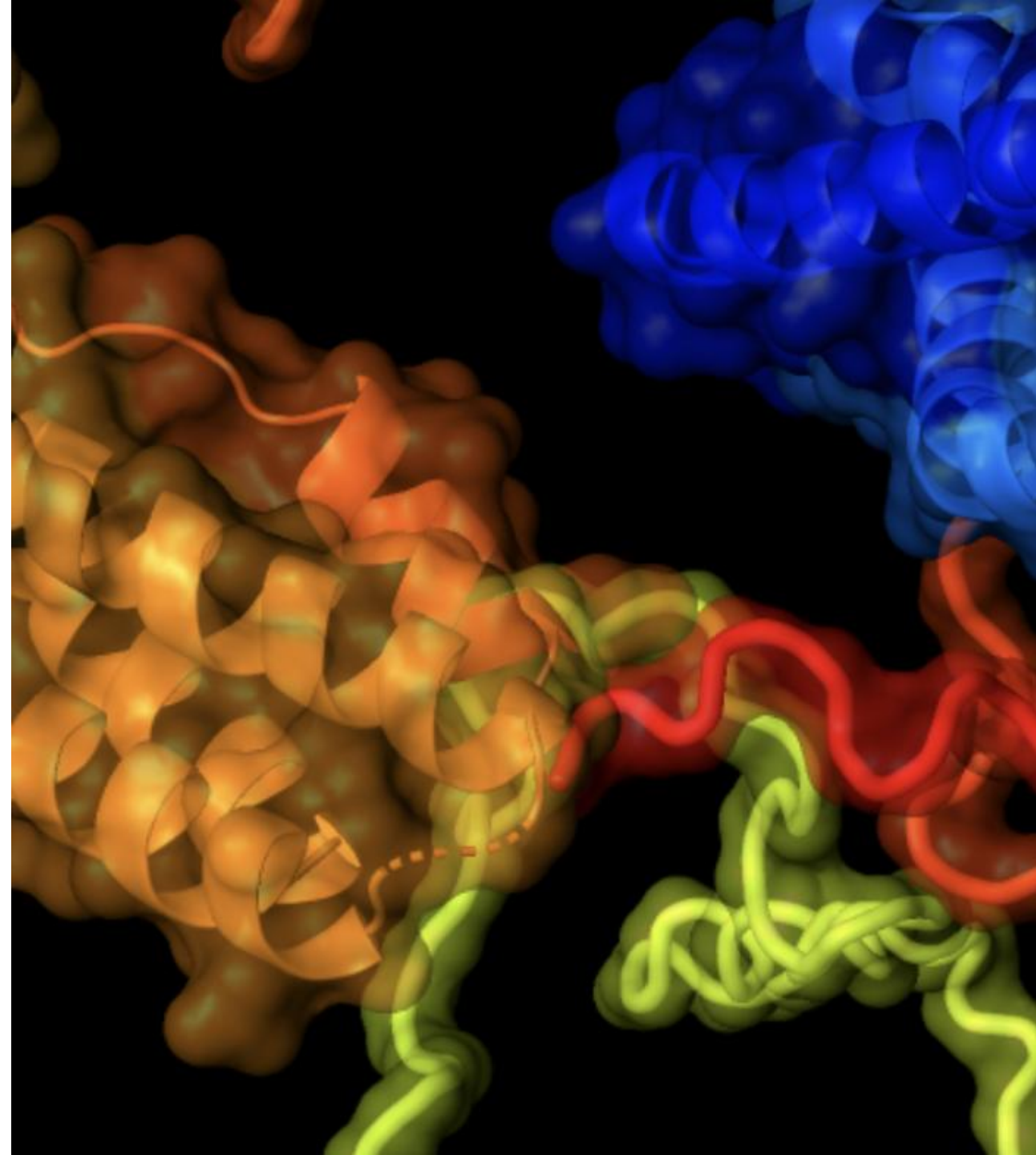
a Local message passing in Geometric GNNs



b Examples of long-range allosteric regulation

Agenda

- Background
- Problem to Solve
- **Method**
- Experiments
- Theoretical Proof
- Conclusion & Discussion



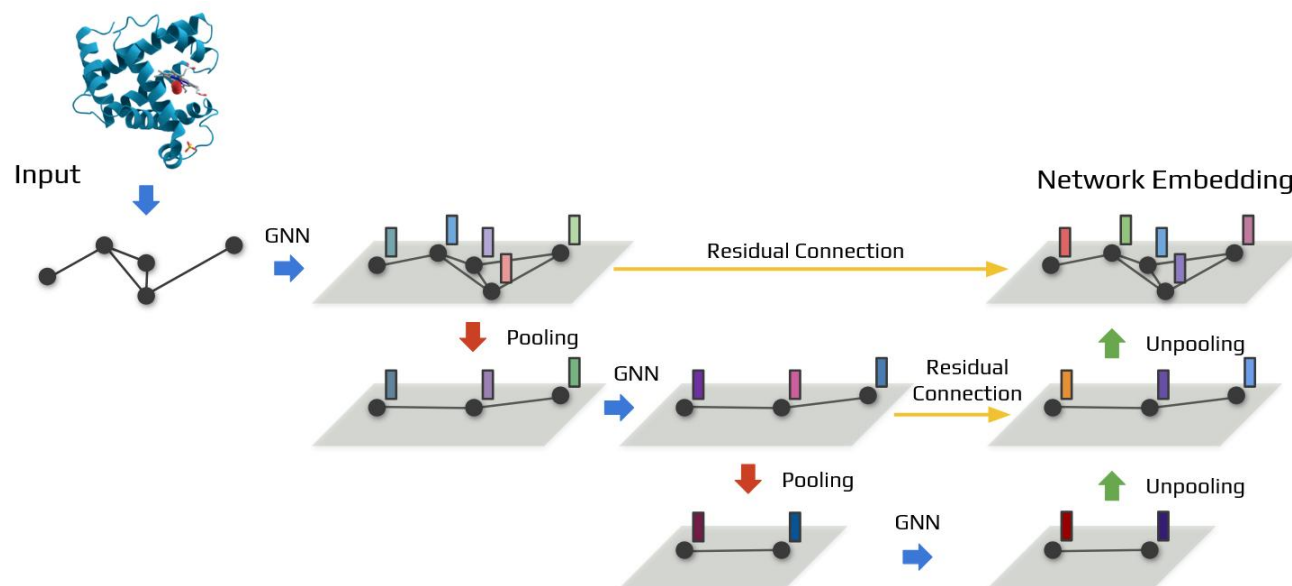
Motivation

We argue that the network architecture itself should mirror this biological hierarchy.

Prior works in hierarchical geometric GNNs can fall into 3 categories:

- 1) Local pooling
- 2) Virtual nodes
- 3) Virtual edges

We introduce **Geometric Graph U-Nets**, a new class of models that learn multi-scale representations by recursively coarsening and refining the protein graph.



Overview

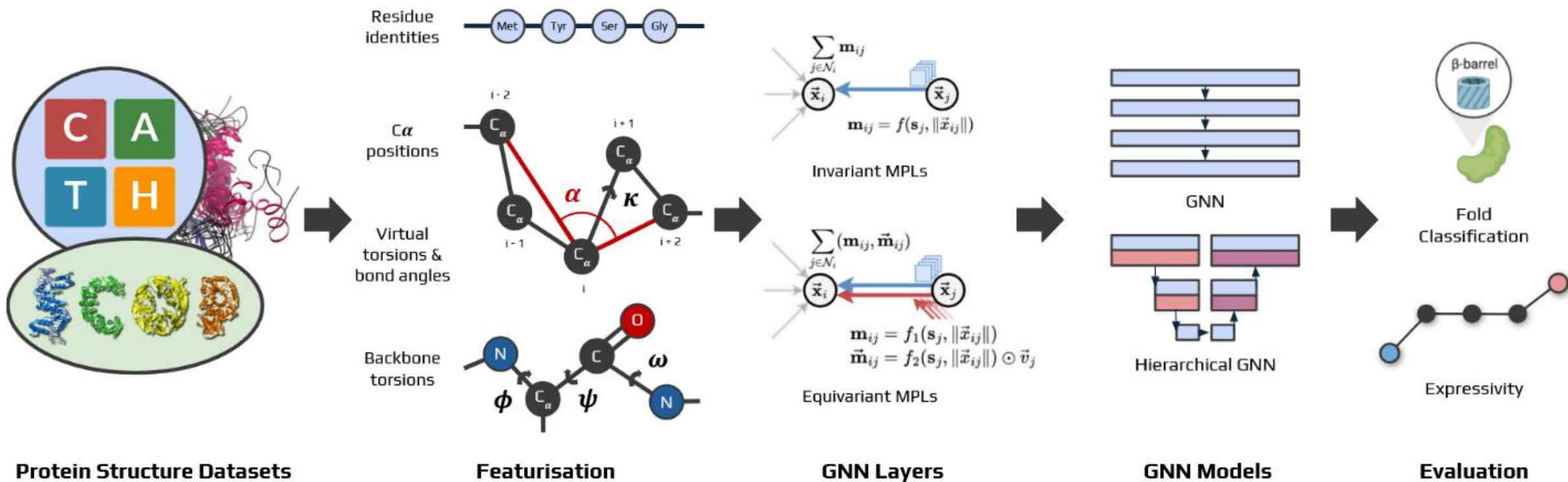


Figure. The overall pipeline of our method, mainly following ProteinWorkshop [38].

Method

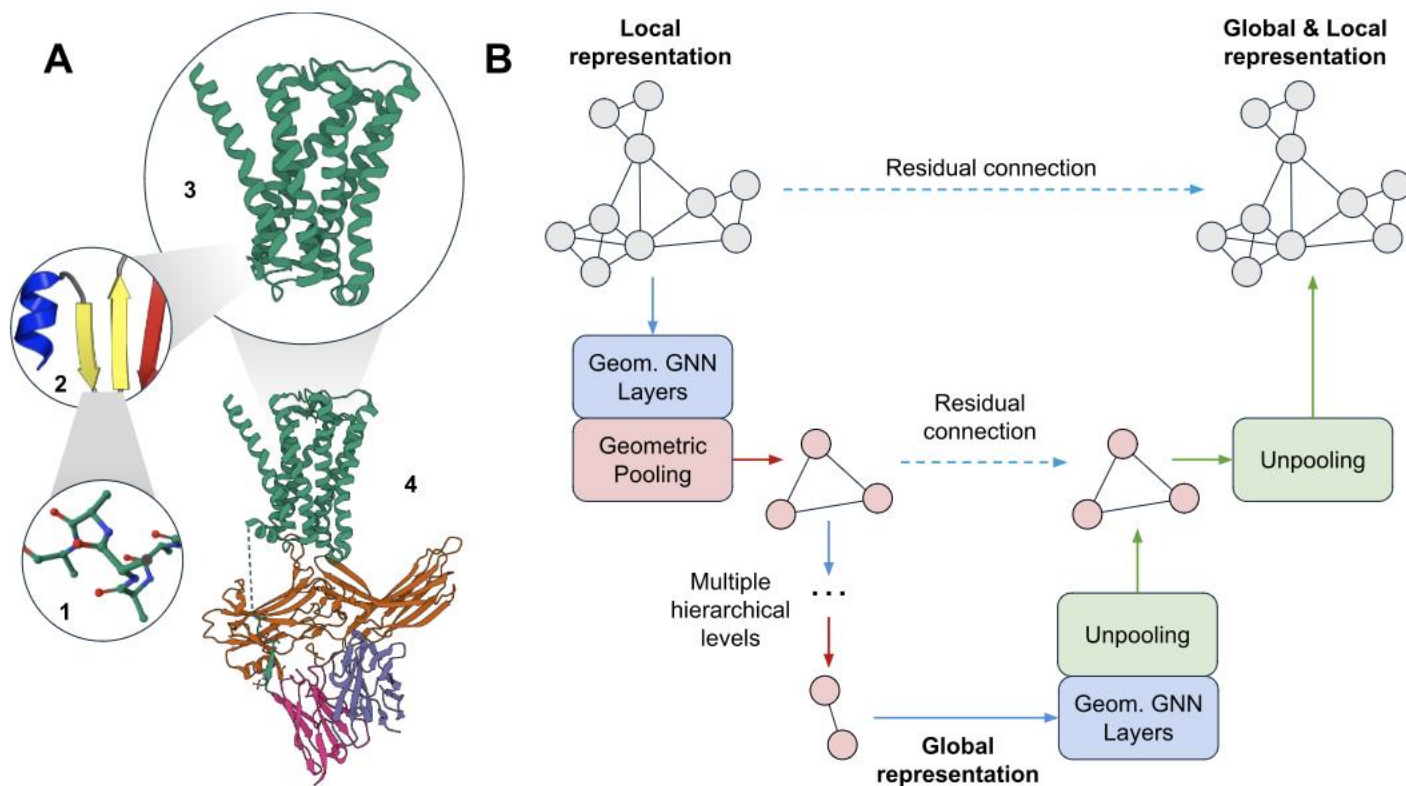


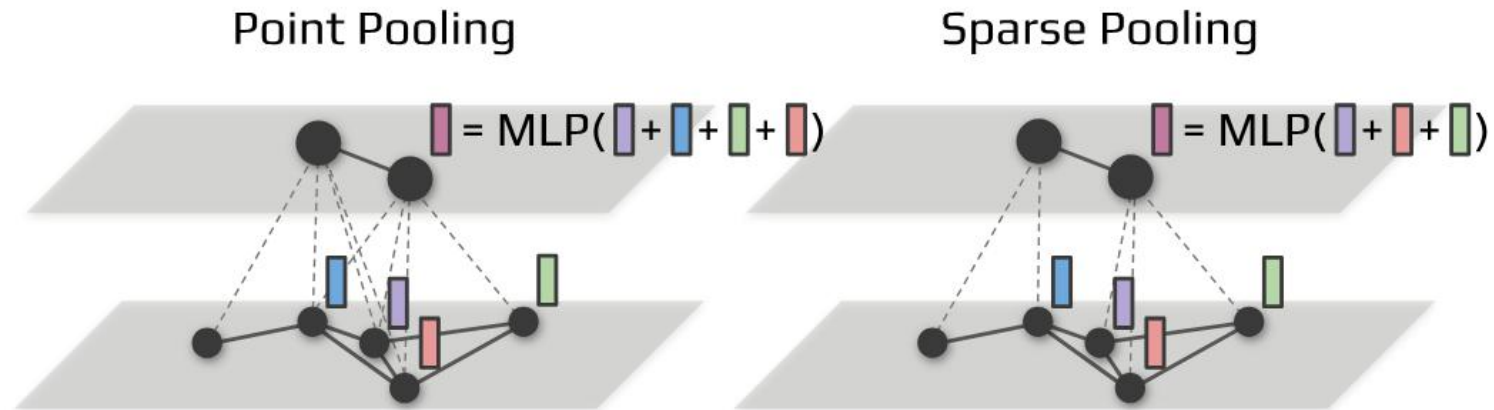
Figure 1: Geometric Graph U-Nets for multi-scale protein representation learning.

(A) Protein function is governed by a structural hierarchy. Standard GNNs struggle to capture these long-range interactions.

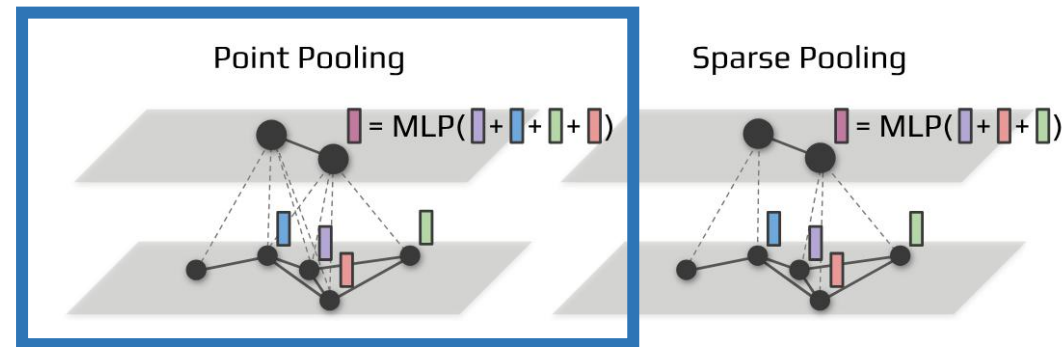
(B) The Geometric Graph U-Net is a biologically inspired architecture that encodes a protein graph by progressive coarsening, moving from local representations to global ones.

Method: Pooling Layers

- Point Pooling Layer vs Sparse Pooling Layer



Point Pooling Layer



Point Pooling Layers adapt the pooling operation used for point cloud data in PointNet++ [21] to protein structures.

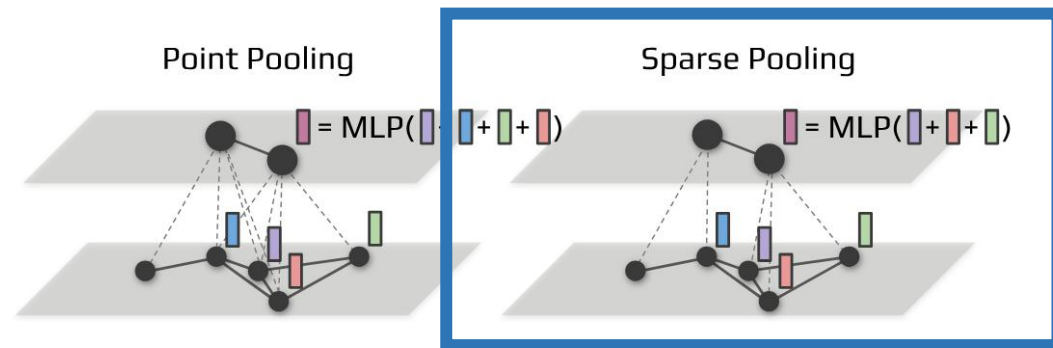
For each pair of nodes $v_i \notin V_P$ and supernodes v_j^{sn} , where v_i denotes the i -th node, V_P stands for the sampled node set, and v_j^{sn} is the j -th supernode, we define the cluster assignment matrix C :

$$C_{j,i} = \begin{cases} 1 & \text{if } i \in \mathcal{N}_j \\ 0 & \text{otherwise} \end{cases}$$

We use the message-passing layer from GIN [35] to update the features of each pooled supernode $j \in C$:

$$\mathbf{s}_j^{sn} = \text{MLP}(C_j^\top \mathbf{S}) = \text{MLP}\left(s_j + \sum_{i \in \mathcal{N}_j} s_i\right)$$

Sparse Pooling Layer



Sparse Pooling Layers assign each node to its nearest supernode, effectively partitioning the structure into distinct, non-overlapping regions.

Similar to the definitions of the point pooling layer, for each pair of nodes $v_i \notin V_P$ and supernodes $v_j^{sn} \in V_P$ we define cluster assignment matrix C :

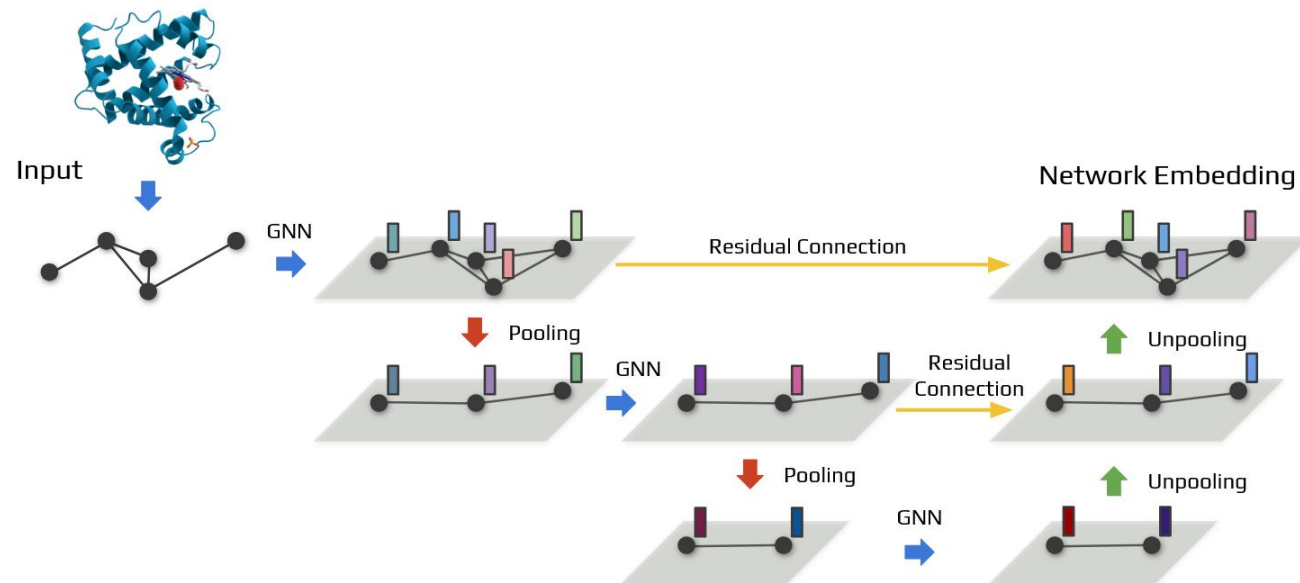
$$C_{j,i} = \begin{cases} 1 & \text{if } i = \arg \min_{k \neq j} \|\mathbf{x}_j - \mathbf{x}_k\|_2 \text{ or } i = j \\ 0 & \text{otherwise} \end{cases}$$

For the reduction function, we use the summation of assigned nodes to update the scalar and vector features of supernodes:

$$\mathbf{s}_j^{sn} = \mathbf{C}_j^T \mathbf{S}, \quad \vec{\mathbf{v}}_j^{sn} = \mathbf{C}_j^T \vec{\mathbf{V}}$$

Geometric U-Net

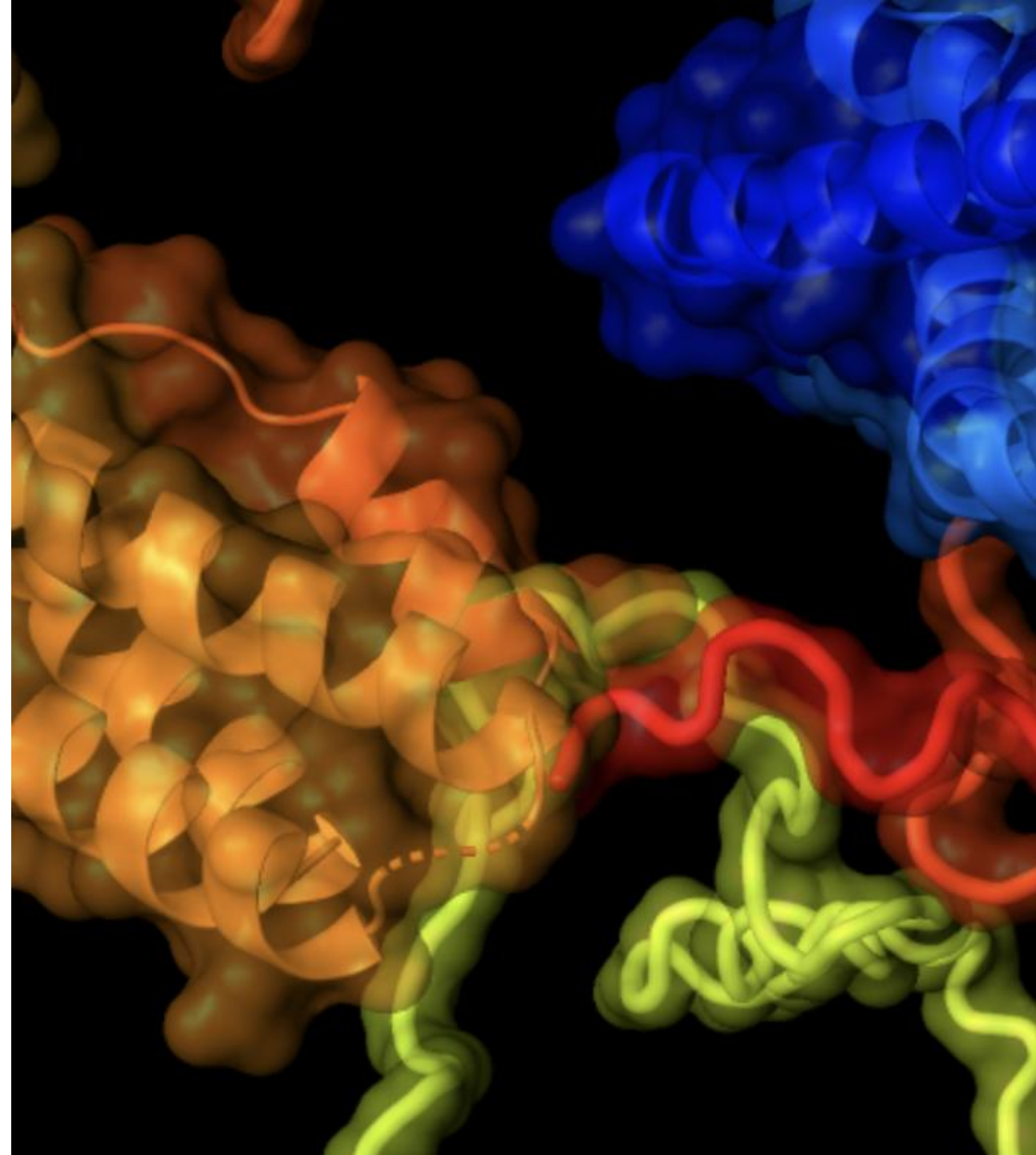
- We extend this encoder in ProteinWorkshop [38] with a hierarchical U-Net architecture comprising encoder and decoder paths with **pooling** and **unpooling** blocks.
- The unpooling block integrates pooled and unpooled features through an unpooling layer.



The unpooled graph is reconstructed with a **skip connection** concatenating cached features from the corresponding pooling blocks to the unpooling output.

Agenda

- Background
- Problem to Solve
- Method
- **Experiments**
- Theoretical Proof
- Conclusion & Discussion



Experiments: Fold Classification

- **Reference:** ProteinWorkshop [38]
- **Dataset:** the SCOP 1.75 dataset
- **System & Hardware:** Ubuntu, Pytorch, and 1x NVIDIA RTX 4090 GPU
- **Training Settings:** < 150 epochs using Adam and learning rates in [0.001, 0.01]
- **Metrics:** accuracy and F1 score

Experiments: Fold Classification

Table 1: **Benchmarking Geometric Graph U-Nets on protein fold classification.** Hierarchical pooling with U-Net architectures consistently improves performance across both invariant (SchNet) and equivariant (GVP) backbones without significantly increasing model parameters. The best results are highlighted in **bold**, with improved performance from the baseline indicated in **green**.

Model	Architecture	Params	Accuracy			F1		
			Fold	Superfamily	Family	Fold	Superfamily	Family
SchNet	Baseline	5.23M	0.331 $\pm$ 0.014	0.442 $\pm$ 0.012	0.928 $\pm$ 0.013	0.087 $\pm$ 0.008	0.157 $\pm$ 0.005	0.688 $\pm$ 0.037
	U-Net Point Pool	5.23M	0.314 $\pm$ 0.007	0.436 $\pm$ 0.014	0.937 $\pm$ 0.007	0.081 $\pm$ 0.001	0.156 $\pm$ 0.014	0.715 $\pm$ 0.033
	U-Net Sparse Pool	5.23M	0.340 $\pm$ 0.010	0.461 $\pm$ 0.002	0.948 $\pm$ 0.003	0.095 $\pm$ 0.007	0.170 $\pm$ 0.004	0.759 $\pm$ 0.018
GVP	Baseline	3.60M	0.336 $\pm$ 0.024	0.512 $\pm$ 0.021	0.961 $\pm$ 0.005	0.097 $\pm$ 0.010	0.215 $\pm$ 0.010	0.793 $\pm$ 0.013
	U-Net Point Pool	3.06M	0.404 $\pm$ 0.002	0.514 $\pm$ 0.007	0.971 $\pm$ 0.003	0.119 $\pm$ 0.002	0.206 $\pm$ 0.001	0.849 $\pm$ 0.010
	U-Net Sparse Pool	3.06M	0.373 $\pm$ 0.015	0.529 $\pm$ 0.009	0.966 $\pm$ 0.004	0.111 $\pm$ 0.005	0.217 $\pm$ 0.004	0.830 $\pm$ 0.019

- Results are reported on three test subsets (Fold, Superfamily, Family), which classify proteins into **1,195** folds. Numbers are averaged through 3 random runs.
- Baselines are tuned with decoders with fewer hidden layers but higher performance.
- **U-Net Point Pool** stands for our U-Net variant model with point pooling layers.
- **U-Net Sparse Pool** stands for our U-Net variant model with sparse pooling layers.

Synthetic Expressivity Using k-chains

- This experiment explores how different numbers of pooling layers enhance expressivity on synthetic k-chain graphs from Joshi et al. [8].
- The GWL (Geometric Weisfeiler–Lehman) test requires $(\lfloor k/2 \rfloor + 1)$ iterations to differentiate k-chains.
- We hope to see whether:
 - 1) all models can distinguish k-chains with fewer layers than the minimal requirement.
 - 2) pooling can enhance expressivity.

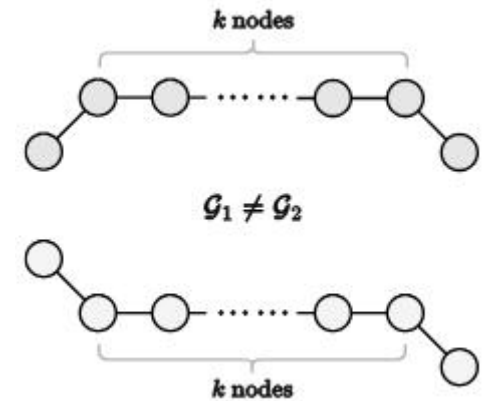


Fig. Illustration of k-chains.

Experiments: Synthetic Expressivity k-chains

Table 2: **k-chain geometric graphs**. Geometric pooling enables all GNN architectures to distinguish k-chains with fewer layers than theoretically required (3 layers), empirically validating Theorem 4.2 that pooling increases expressivity. k -chains are $(\lfloor \frac{k}{2} \rfloor + 1)$ -hop distinguishable and $(\lfloor \frac{k}{2} \rfloor + 1)$ GWL iterations are theoretically sufficient to distinguish them. Anomalous results are marked in red and expected results in green.

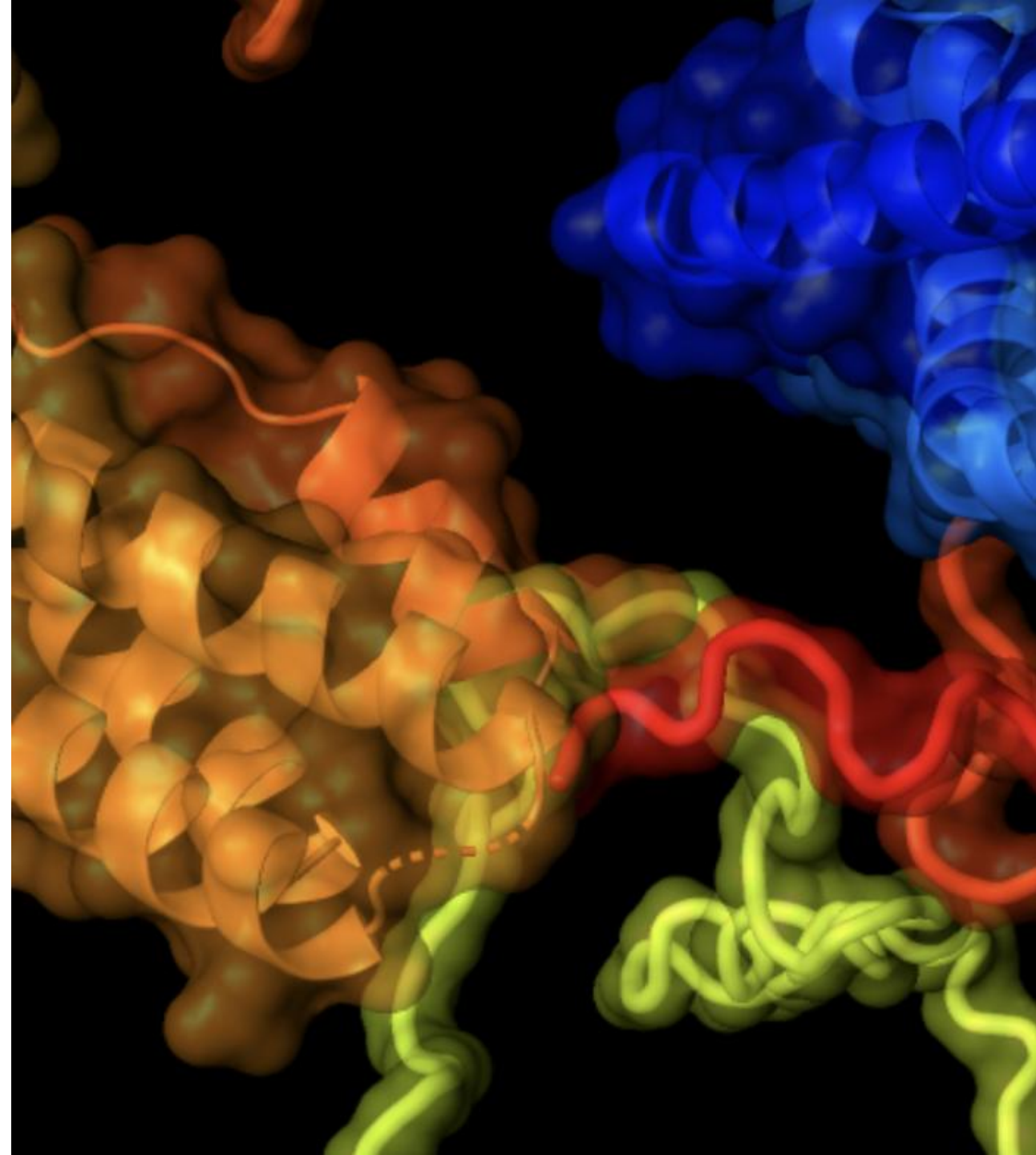
$(k = 4\text{-chains})$		Number of layers				
GNN Layer		$\lfloor \frac{k}{2} \rfloor$	$\lfloor \frac{k}{2} \rfloor + 1 = 3$	$\lfloor \frac{k}{2} \rfloor + 2$	$\lfloor \frac{k}{2} \rfloor + 3$	$\lfloor \frac{k}{2} \rfloor + 4$
Equivariant	GWL	50%	100%	100%	100%	100%
	E-GNN	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	100.0 $\pm$ 0.0
	E-GNN - 3 nodes	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
	E-GNN - 4 nodes	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
	GVP-GNN	50.0 $\pm$ 0.0	90.0 $\pm$ 20.0	80.0 $\pm$ 24.5	100.0 $\pm$ 0.0	90.0 $\pm$ 20.0
	GVP-GNN - 3 nodes	100.0 $\pm$ 0.0	95.0 $\pm$ 15.0	100.0 $\pm$ 0.0	90.0 $\pm$ 20.0	100.0 $\pm$ 0.0
	GVP-GNN - 4 nodes	100.0 $\pm$ 0.0	95.0 $\pm$ 15.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
	TFN	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0
	TFN - 3 nodes	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
	TFN - 4 nodes	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
Invariant	IGWL	50%	50%	50%	50%	50%
	SchNet	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0
	SchNet - 3 nodes	80.0 $\pm$ 24.5	95.0 $\pm$ 15.0	90.0 $\pm$ 20.0	95.0 $\pm$ 15.0	95.0 $\pm$ 15.0
	SchNet - 4 nodes	85.0 $\pm$ 22.9	95.0 $\pm$ 15.0	95.0 $\pm$ 15.0	100.0 $\pm$ 0.0	95.0 $\pm$ 15.0
	SphereNet	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0	50.0 $\pm$ 0.0
	SphereNet - 3 nodes	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
	SphereNet - 4 nodes	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0

Theoretical Proof

- First, we study expressivity through the lens of distinguishing non-isomorphic geometric graphs, adopting the Geometric Weisfeiler–Lehman (GWL) test [8].
- Then, we extend the hierarchical expressivity framework introduced for pooling to the geometric setting, enabling analysis of when pooling preserves or strengthens GWL distinguishing capacity.
- **POOL maintains expressivity.**

Agenda

- Background
- Problem to Solve
- Method
- Experiments
- Theoretical Proof
- **Conclusion & Discussion**



Conclusion & Discussion

- The architecture of Geometric GNNs should reflect the hierarchical nature of the biological structures.
- Our Geometric Graph U-Net serves as a flexible blueprint for a new generation of multi-scale models.
- By treating proteins as nested structural assemblies, we show how multi-scale architectures can yield more powerful representations for predicting function from structure.
- Further validation on a broader range of structure-function prediction tasks, e.g. inverse folding.
- Modelling biomolecular complexes with hierarchical interactions across multiple scales.
- Particularly, we believe Geometric U-Nets may be especially effective as encoders for multi-state inverse design pipelines [47, 48].

Thank you!

